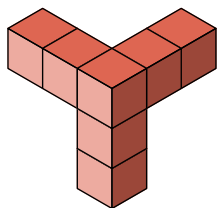
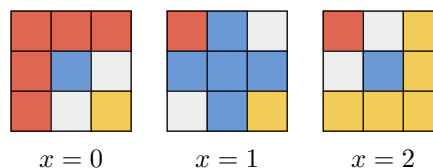


September 6, 2026 at 10:05

1. Introduction. Exercise 7.2.2.1–346 is about tripods. An (l, m, n) -tripod is a corner cubie with three legs of l , m and n cubies attached to it, and the exercise asks how much of space you can fill with shifted copies of one—no turning, no reflecting, only translation. Five parts: (a) the $(1, m, n)$ -tripods tile space exactly; (b) the $(2, 2, 2)$ -tripods reach $7/9$ of it; (c) the $(3, 3, 3)$ -tripods reach $65/108$; (d) a packing of “pods” in a cuboid can be doubled into a packing of a torus by tripods; and (e) Algorithm M evaluates $r(l, m, n)$, the number of pods that fit, for $4 \leq l \leq m \leq n \leq 6$.

This program checks all of it. Everything the answer prints comes out right, and the last two parts pick up something the answer does not say: the $7/9$ of part (b) is already reached by three tripods with period 3, and the same construction with period 4 gives $5/8$ for part (c), which beats $65/108$.

the $(2, 2, 2)$ -tripod

three of them fill 21 of the 27 cubies

```
package main
import (
    "context"
    "flag"
    "fmt"
    "io"
    "sort"
    "strconv"
    "strings"
    "time"
    cells "github.com/sjnam/dancing-cells"
)
< Types 4 >
< Functions 5 >
func main() {
    < Read the command line 2 >
    < Do what the mode asks 3 >
}
```

2. $\langle$ Read the command line 2 $\rangle \equiv$

```

mode := flag.String("mode", "all", "a, b, c, diag, d, e, torus, or all")
top := flag.Int("top", 12, "largest leg to try in part (a) and in the diagonals")
legs := flag.String("legs", "2,2,2", "which tripod the torus sweep is for")
vol := flag.Int("vol", 120, "largest torus volume to sweep")
mins := flag.Int("mins", 60, "minutes allowed for any one search")
flag.Parse()
budget := time.Duration(*mins) * time.Minute
var sweep vec
fmt.Sscanf(*legs, "%d,%d,%d", &sweep.x, &sweep.y, &sweep.z)

```

This code is used in section 1.

3. $\langle$ Do what the mode asks 3 $\rangle \equiv$

```

if *mode == "a" ∨ *mode == "all" {
     $\langle$  Tile space with  $(1, m, n)$ -tripods 11  $\rangle$ 
}
if *mode == "b" ∨ *mode == "all" {
     $\langle$  Read the packing printed in answer 346(b) 14  $\rangle$ 
}
if *mode == "c" ∨ *mode == "all" {
     $\langle$  Check the thirteen corners of answer 346(c) 17  $\rangle$ 
}
if *mode == "diag" ∨ *mode == "all" {
     $\langle$  Check the diagonal packings 19  $\rangle$ 
}
if *mode == "d" ∨ *mode == "all" {
     $\langle$  Double a pod packing as part (d) says 28  $\rangle$ 
}
if *mode == "e" ∨ *mode == "all" {
     $\langle$  Evaluate  $r(l, m, n)$  26  $\rangle$ 
}
if *mode == "torus" {
     $\langle$  Sweep the tori 31  $\rangle$ 
}

```

This code is used in section 1.

4. Tripods, pods, and tori. A cubie is a point of $\mathbf{Z}^3$, and a tripod is a list of them. The legs run towards *smaller* coordinates, because that is how the exercise defines a pod: the pod at (l, m, n) is

$$\{(l, m, n)\} \cup \{(l', m, n) \mid 0 \leq l' < l\} \cup \{(l, m', n) \mid 0 \leq m' < m\} \cup \{(l, m, n') \mid 0 \leq n' < n\},$$

so its three legs reach exactly back to the coordinate planes and it is the (l, m, n) -tripod in that one position.

⟨Types 4⟩ ≡

```
type vec struct { x, y, z int }
```

This code is used in section 1.

5. Most of the work happens in a torus, where the coordinates are read modulo some $X \times Y \times Z$. Passing a zero for a component means “do not wrap in that direction”, which is how the checks in plain $\mathbf{Z}^3$ are done.

⟨Functions 5⟩ ≡

```
func wrap(v, t vec) vec {
  if t.x > 0 {
    v.x = ((v.x % t.x) + t.x) % t.x
  }
  if t.y > 0 {
    v.y = ((v.y % t.y) + t.y) % t.y
  }
  if t.z > 0 {
    v.z = ((v.z % t.z) + t.z) % t.z
  }
  return v
}
```

This code is also defined in sections 6, 7, 8, 9, 10, 13, 22, 25, 30.

This code is used in section 1.

6. Here is the tripod itself: the corner, and three legs of the lengths *leg* asks for.

⟨Functions 5⟩ +≡

```
func tripod(c, leg, t vec) []vec {
  s := []vec{wrap(c, t)}
  for i := 1; i ≤ leg.x; i++ {
    s = append(s, wrap(vec{c.x - i, c.y, c.z}, t))
  }
  for j := 1; j ≤ leg.y; j++ {
    s = append(s, wrap(vec{c.x, c.y - j, c.z}, t))
  }
  for k := 1; k ≤ leg.z; k++ {
    s = append(s, wrap(vec{c.x, c.y, c.z - k}, t))
  }
  return s
}
```

7. Laying pieces down one at a time and complaining about collisions is the only test any of this needs. The cubies covered come back with the count of collisions, since some of the checks want to look at them afterwards.

⟨Functions 5⟩ +≡

```

func lay(pieces [][]vec)(used map[vec]bool, clashes int) {
    used = map[vec]bool{ }
    for _, p := range pieces {
        for _, c := range p {
            if used[c] {
                clashes++
            }
            used[c] = true
        }
    }
    return used, clashes
}

```

8. Cubies become item names, and come back from a solution the same way.

⟨Functions 5⟩ +≡

```

func name(v vec) string { return fmt.Sprintf("%d.%d.%d", v.x, v.y, v.z) }
func parse(s string) vec {
    p := strings.Split(s, ".")
    x, _ := strconv.Atoi(p[0])
    y, _ := strconv.Atoi(p[1])
    z, _ := strconv.Atoi(p[2])
    return vec{x, y, z}
}

```

9. Every search here asks the same question—is there a packing with t pieces?—so one wrapper does for all of them. It returns the first solution, and says whether it ran out of time rather than out of possibilities.

⟨Functions 5⟩ +≡

```

func first(r io.Reader, budget time.Duration)(sol []cells.Option, slow bool) {
    ctx, cancel := context.WithTimeout(context.Background(), budget)
    defer cancel()
    res := cells.NewMCC().WithContext(ctx).Dance(r)
    for s := range res.Solutions {
        sol = append(sol, s...)
        cancel()
        for range res.Solutions {
        }
        return sol, false
    }
    return  $\Lambda$ , ctx.Err()  $\neq$   $\Lambda$ 
}

```

10. Part (a): tiling space with $(1, m, n)$ -tripods. The hint says to pack N^2 tripods into an $N \times N \times N$ torus, where $N = m + n + 2$. Since a $(1, m, n)$ -tripod has $1 + 1 + m + n = N$ cubies, that is a perfect tiling if it works at all, and a tiling of the torus is a tiling of space.

Follow the hint. Take the N tripods whose corners are $(i, t + \delta, t)$ for $t = 0, 1, \dots, N - 1$: a family on layer i , shifted by multiples of $(0, 1, 1)$ as the answer says. Sort the cubies of layer i by the “diagonal” they lie on, meaning $(y - z) \bmod N$. The corner of tripod t sits on diagonal δ ; its y -leg reaches diagonals $\delta - 1$ through $\delta - m$; its z -leg reaches $\delta + 1$ through $\delta + n$. As t runs over its N values each of those $1 + m + n = N - 1$ diagonals fills up completely, so the family covers every cubie of layer i except the single diagonal $\delta + n + 1$. Meanwhile the leg of length 1 drops one cubie onto layer $i - 1$, on diagonal δ .

So the families fit together exactly when the diagonal that layer $i + 1$ drops is the diagonal that layer i is missing, that is when $\delta_{i+1} = \delta_i + n + 1$. Setting $\delta_i = i(n + 1) \bmod N$ does it, and closes up around the torus for free. Here it is.

⟨ Functions 5 ⟩ $+\equiv$

```
func layerA(i, m, n int) [][]vec {
    N := m + n + 2
    d := (i * (n + 1)) % N
    var fam [][]vec
    for t := range N {
        c := vec{i, (t + d) % N, t}
        fam = append(fam, tripod(c, vec{1, m, n}, vec{N, N, N}))
    }
    return fam
}
```

11. The test is the whole torus at once: N^2 tripods, N^3 cubies, no collisions.

⟨ Tile space with $(1, m, n)$ -tripods 11 ⟩ $\equiv$

```
fmt.Println("(a) shifted (1,m,n)-tripods tile the (m+n+2)^3 torus")
bad := 0
for m := 0; m < *top; m++ {
    for n := 0; n < *top; n++ {
        N := m + n + 2
        var all [][]vec
        for i := range N {
            all = append(all, layerA(i, m, n)...)
        }
        used, clashes := lay(all)
        if len(all) != N * N || len(used) != N * N * N || clashes != 0 {
            bad++
            fmt.Printf("(1,%d,%d): %d tripods, %d cubies of %d, %d clashes\n",
                m, n, len(all), len(used), N * N * N, clashes)
        }
    }
}
fmt.Printf("%d pairs (m,n) with 0 <= m, n <= %d: %d failures\n",
    (*top + 1) * (*top + 1), *top, bad)
```

⟨ Check the two facts the construction rests on 12 ⟩

This code is used in section 3.

12. It is worth watching the argument itself work, not just its conclusion. For one family I ask which diagonal of its own layer it leaves alone, and which diagonal it drops on the layer below; the first should be $n + 1$ ahead of the second.

⟨ Check the two facts the construction rests on 12 ⟩ $\equiv$

```

bad = 0
for m := 0; m ≤ *top; m++ {
  for n := 0; n ≤ *top; n++ {
    N := m + n + 2
    fam := layerA(1, m, n)
    here, below := map[int]int{ }, map[int]int{ }
    for _, p := range fam {
      for _, c := range p {
        if c.x ≡ 1 {
          here[((c.y - c.z) % N + N) % N]++
        } else {
          below[((c.y - c.z) % N + N) % N]++
        }
      }
    }
    miss, drop := -1, -1
    for d := range N {
      if here[d] ≡ 0 {
        miss = d
      }
      if below[d] ≡ N {
        drop = d
      }
    }
    if miss < 0 ∨ drop < 0 ∨ (drop + n + 1) % N ≠ miss ∨ len(below) ≠ 1 {
      bad++
      fmt.Printf("␣(1,%d,%d):␣misses␣%d,␣drops␣%d\n", m, n, miss, drop)
    }
  }
}
fmt.Printf("␣each␣family␣covers␣all␣of␣its␣layer␣but␣one␣diagonal," +
  "␣and␣drops␣one␣whole␣diagonal:␣%d␣failures\n", bad)

```

This code is used in section 11.

13. Part (b): the packing that is printed, and 7/9. Answer 346(b) draws twelve (2, 2, 2)-tripods in a $3 \times 6 \times 6$ torus as three 6×6 layers, one symbol to a piece. Twelve pieces of seven cubies is 84 of the 108 cubies, which is the 7/9 the exercise asks for. I copied the three layers out of the book.

⟨ Functions 5 ⟩ +≡

```
func printedB() [3][6]string {
    return [3][6]string{
        {"012600", "112371", "222348", "933345", "0a4445", "01b555"},
        {"066678", "917778", "9a2888", "9ab399", "aab64a", "bbb675"},
        {"0..6..", ".1..7.", "..2..8", "9..3..", ".a..4.", "..b..5"},
    }
}
```

14. Each symbol must name a genuine (2, 2, 2)-tripod, which I test by looking for a cubie of the piece whose tripod is the piece. The three long directions of the torus are 3, 6 and 6, and a leg of length 2 in the direction of length 3 sweeps a whole line, so a piece shows up in all three layers at one place and in its corner's layer at four more.

⟨ Read the packing printed in answer 346(b) 14 ⟩ ≡

```
fmt.Println("(b) the twelve tripods printed in answer 346(b)")
t := vec{3, 6, 6}
piece := map[byte][]vec{}
for x, layer := range printedB() {
    for y, row := range layer {
        for z := range 6 {
            if ch := row[z]; ch != '.' {
                piece[ch] = append(piece[ch], vec{x, y, z})
            }
        }
    }
}
```

⟨ Name the corner of every printed piece 15 ⟩

This code is used in section 3.

15. $\langle$ Name the corner of every printed piece 15 $\rangle \equiv$

```

var syms []int
for ch := range piece {
    syms = append(syms, int(ch))
}
sort.Ints(syms)
var all [][]vec
bad := 0
for _, s := range syms {
    cs := piece[byte(s)]
    corner, found := vec{ }, false
    for _, c := range cs {
         $\langle$  Ask whether the tripod at  $c$  is exactly the piece 16  $\rangle$ 
        if match {
            corner, found = c, true
        }
    }
    if !found {
        bad++
        fmt.Printf("%%piece%c is not a (2,2,2)-tripod\n", byte(s))
        continue
    }
    all = append(all, cs)
    fmt.Printf("%%piece%c: corner (%d,%d,%d)\n", byte(s), corner.x, corner.y, corner.z)
}
used, clashes := lay(all)
fmt.Printf("%%pieces, %%d cubies of %%d, %%d clashes, %%d not tripods\n",
    len(all), len(used), 3*6*6, clashes, bad)
fmt.Printf("%%density %%d/%%d=%%.6f%%(7/9=%%.6f)\n",
    len(used), 108, float64(len(used))/108, 7.0/9)

```

This code is used in section 14.

16. The piece is a translate of the tripod exactly when some cubie of it is a corner whose tripod has the piece's cubies and no others.

$\langle$ Ask whether the tripod at c is exactly the piece 16 $\rangle \equiv$

```

in := map[vec]bool{ }
for _, q := range tripod(c, vec{2,2,2}, t) {
    in[q] = true
}
match := len(in) == len(cs)
for _, q := range cs {
    if !in[q] {
        match = false
    }
}

```

This code is used in section 15.

17. Part (c): thirteen tripods, and a better packing. Answer 346(c) places thirteen $(3, 3, 3)$ -tripods in a $6 \times 6 \times 6$ torus and lists their corners. Thirteen pieces of ten cubies is 130 of 216, which is the $65/108$ the exercise claims.

$\langle$ Check the thirteen corners of answer 346(c) 17 $\rangle \equiv$

```
fmt.Println("(c) the thirteen corners listed in answer 346(c)")
corners := []vec{{0, 0, 0}, {0, 1, 1}, {0, 2, 2}, {1, 1, 3}, {1, 2, 4},
  {2, 3, 2}, {2, 4, 4}, {3, 3, 3}, {3, 4, 5}, {4, 4, 0}, {4, 5, 1},
  {5, 0, 5}, {5, 5, 3}}
var all [][]vec
for _, c := range corners {
  all = append(all, tripod(c, vec{3, 3, 3}, vec{6, 6, 6}))
}
used, clashes := lay(all)
fmt.Printf("%d tripods, %d cubies of %d, %d clashes\n",
  len(all), len(used), 216, clashes)
fmt.Printf("density %d/%d = %.6f (65/108 = %.6f)\n",
  len(used), 216, float64(len(used))/216, 65.0/108)
```

This code is used in section 3.

18. The diagonal packing. Both (b) and (c) are instances of one construction, and it is easier than either of them. Work in the $(n+1)^3$ torus and take the $n+1$ tripods whose corners run down the main diagonal, at (p, p, p) for $0 \leq p \leq n$. A leg of length n in a direction of length $n+1$ sweeps a whole line, so the tripod at (p, p, p) is exactly the set of cubies with at least two coordinates equal to p .

Two of them can never meet: a cubie in both would need two of its three coordinates equal to p and two equal to q , and three coordinates cannot do that for $p \neq q$. So the $n+1$ tripods are disjoint for nothing, and the cubies they miss are the ones whose three coordinates are all different, of which there are $(n+1)n(n-1)$. The density is

$$1 - \frac{n(n-1)}{(n+1)^2} = \frac{3n+1}{(n+1)^2},$$

which is 1 at $n=1$, $7/9$ at $n=2$ —answer (b)’s number, from three pieces of period 3 rather than twelve in a $3 \times 6 \times 6$ torus—and $5/8$ at $n=3$, which is more than answer (c)’s $65/108$.

19. I check this in $\mathbf{Z}^3$ with no torus arithmetic at all: lay out every tripod whose corner is $(p, p, p) + (n+1)(a, b, c)$ for $a, b, c \leq 3$ and see whether any two of the thousands of them collide.

⟨ Check the diagonal packings 19 ⟩ ≡

```
fmt.Println("diagonal_packing: n+1 tripods with corners (p,p,p), period n+1")
for n := 1; n ≤ *top; n++ {
  N := n + 1
  var all [][]vec
  for a := -3; a ≤ 3; a++ {
    for b := -3; b ≤ 3; b++ {
      for c := -3; c ≤ 3; c++ {
        for p := range N {
          corner := vec{p + N * a, p + N * b, p + N * c}
          all = append(all, tripod(corner, vec{n, n, n}, vec{ }))
        }
      }
    }
  }
  used, clashes := lay(all)
  ⟨ Measure the density inside a window 20 ⟩
  ⟨ Report the density of the diagonal packing 21 ⟩
}
```

This code is used in section 3.

20. The formula for the density is one thing; counting is another. I take a window well inside the block that was laid out—far enough from its edges that no tripod is missing—and ask what fraction of it is covered. That number owes nothing to the argument it is testing.

⟨Measure the density inside a window 20⟩ ≡

```

lo, hi := -2 * N, 2 * N
inside, filled := 0, 0
for x := lo; x < hi; x++ {
  for y := lo; y < hi; y++ {
    for z := lo; z < hi; z++ {
      inside++
      if used[vec{x, y, z}] {
        filled++
      }
    }
  }
}

```

This code is used in section 19.

21. The density is read off the torus, where $N^3 - Nn(n-1)$ cubies out of N^3 are covered, and the note at the end says which part of the answer it speaks to.

⟨Report the density of the diagonal packing 21⟩ ≡

```

num, den := N * N * N - N * n * (n - 1), N * N * N
note := ""
if n == 2 {
  note = "answer346(b)'s 7/9"
}
if n == 3 {
  note = fmt.Sprintf("answer346(c)'s 65/108=%.6f", 65.0/108)
}
fmt.Printf("n=%2d: %5d tripods laid, %d clashes; density %d/%d=%.6f, " +
  "counted %d/%d=%.6f\n",
  n, len(all), clashes, num, den, float64(num)/float64(den),
  filled, inside, float64(filled)/float64(inside), note)

```

This code is used in section 19.

22. Part (e): how many pods fit in a cuboid. Answer 346(e) gives the encoding: one primary item # of multiplicity t , one secondary item per cubie of the $l \times m \times n$ cuboid, and one option per pod. Because every cubie is secondary, a solution is nothing but a set of t pods no two of which share a cubie, and asking for t of them is what the multiplicity is for. The answer adds a speed-up: make the cubies ${}^{\circ}00$ and $(l-1)(m-1)(n-1)$ primary too, “because those two pods can be assumed to be present”.

⟨ Functions 5 ⟩ +≡

```

func podProblem( $l, m, n, t$  int,  $force$  bool) io.Reader {
     $r, w := io.Pipe()$ 
     $far := \text{vec}\{l-1, m-1, n-1\}$ 
    go func() {
        defer  $w.Close()$ 
         $fmt.Fprintf(w, "\%d:\%d\#\", t, t)$ 
        if  $force$  {
             $fmt.Fprintf(w, "\%s\%", name(\text{vec}\{ \}), name(far))$ 
        }
         $fmt.Fprint(w, "\%|\")$ 
        ⟨ Name the cubies that stay secondary 23 ⟩
         $fmt.Fprintln(w)$ 
        ⟨ Write one option for each pod 24 ⟩
    }()
    return  $r$ 
}

```

23. ⟨ Name the cubies that stay secondary 23 ⟩ ≡

```

for  $x := \text{range } l$  {
    for  $y := \text{range } m$  {
        for  $z := \text{range } n$  {
             $c := \text{vec}\{x, y, z\}$ 
            if  $force \wedge (c \equiv (\text{vec}\{ \}) \vee c \equiv far)$  {
                continue
            }
             $fmt.Fprintf(w, "\%s\%", name(c))$ 
        }
    }
}

```

This code is used in section 22.

24. A pod is just the tripod whose legs are as long as its corner is far from the coordinate planes, and it never wraps—so $\text{tripod}(c, c, \text{vec}\{\})$ is the pod at c . The corner comes first in every option, so a solution can be read back by looking at each option's second word.

⟨ Write one option for each pod 24 ⟩ $\equiv$

```

for  $x := \text{range } l$  {
  for  $y := \text{range } m$  {
    for  $z := \text{range } n$  {
       $c := \text{vec}\{x, y, z\}$ 
       $\text{fmt.Fprint}(w, "\#")$ 
      for  $\_, q := \text{range } \text{tripod}(c, c, \text{vec}\{\})$  {
         $\text{fmt.Fprintf}(w, "\_ \%s", \text{name}(q))$ 
      }
       $\text{fmt.Fprintln}(w)$ 
    }
  }
}

```

This code is used in section 22.

25. To find $r(l, m, n)$ I ask for more and more pods until the answer is no. With the two cubies forced the count has to start at two, since the far corner can be covered by no pod but its own; without them it starts at one. Either way a packing of t pods yields one of $t - 1$, so the first refusal is the answer.

⟨ Functions 5 ⟩ $+\equiv$

```

func  $rOf(l, m, n \text{ int}, \text{force } \text{bool}, \text{budget } \text{time.Duration})(\text{int}, []\text{vec}, \text{bool}) \{$ 
   $\text{force} = \text{force} \wedge l * m * n > 1$ 
   $t := 1$ 
  if  $\text{force}$  {
     $t = 2$ 
  }
  var  $\text{best } []\text{vec}$ 
  for {
     $\text{sol}, \text{slow} := \text{first}(\text{podProblem}(l, m, n, t, \text{force}), \text{budget})$ 
    if  $\text{slow}$  {
      return  $t - 1, \text{best}, \text{true}$ 
    }
    if  $\text{sol} \equiv \Lambda$  {
      return  $t - 1, \text{best}, \text{false}$ 
    }
     $\text{best} = \text{best}[:0]$ 
    for  $\_, \text{opt} := \text{range } \text{sol}$  {
       $\text{best} = \text{append}(\text{best}, \text{parse}(\text{opt}[1]))$ 
    }
     $t++$ 
  }
}

```

26. Ten cuboids, each done twice—with the answer’s speed-up and without it—so that the assumption behind it can be seen not to change any of the answers.

⟨ Evaluate $r(l, m, n)$ 26 ⟩ $\equiv$

```

fmt.Println("(e)␣r(1,m,n)␣for␣4␣≤␣l␣≤␣m␣≤␣n␣≤␣6")
want := map[string]int{"444": 8, "445": 9, "446": 9, "455": 10, "456": 10,
    "466": 12, "555": 11, "556": 12, "566": 13, "666": 14}
for l := 4; l ≤ 6; l++ {
    for m := l; m ≤ 6; m++ {
        for n := m; n ≤ 6; n++ {
            ⟨ Evaluate one cuboid both ways 27 ⟩
        }
    }
}

```

This code is used in section 3.

27. ⟨ Evaluate one cuboid both ways 27 ⟩ $\equiv$

```

key := fmt.Sprintf("%d%d%d", l, m, n)
t0 := time.Now()
r1, _, slow1 := rOf(l, m, n, true, budget)
d1 := time.Since(t0).Round(time.Millisecond)
t0 = time.Now()
r2, _, slow2 := rOf(l, m, n, false, budget)
d2 := time.Since(t0).Round(time.Millisecond)
mark := "ok"
if r1 ≠ want[key] ∨ r2 ≠ want[key] {
    mark = fmt.Sprintf("DIFFERS␣from␣answer␣346(e)'s␣%d", want[key])
}
if slow1 ∨ slow2 {
    mark = "TIMED␣OUT"
}
fmt.Printf("␣%s␣->␣%2d␣forced␣(%8v),␣%2d␣plain␣(%8v)␣␣%s␣\n",
    key, r1, d1, r2, d2, mark)

```

This code is used in section 26.

28. Part (d): doubling a pod packing. Part (d) says that $r(l, m, n)$ pods in an $l \times m \times n$ cuboid give $2r(l, m, n)$ nonoverlapping tripods in a $2l \times 2m \times 2n$ torus: put a tripod corner at every pod corner, and another at every pod corner shifted by (l, m, n) . That is $2r(1 + l + m + n)$ cubies out of $8lmn$, which is the $(1 + l + m + n)r(l, m, n)/(4lmn)$ of the exercise.

I take a maximum pod packing from the machinery of part (e) and try it.

⟨Double a pod packing as part (d) says 28⟩ $\equiv$

```
fmt.Println("(d) 2r(1,m,n) tripods in the 2l x 2m x 2n torus")
for l := 2; l ≤ 4; l++ {
  for m := l; m ≤ 4; m++ {
    for n := m; n ≤ 4; n++ {
      r, corners, _ := rOf(l, m, n, true, budget)
      ⟨Lay the doubled packing in its torus 29⟩
    }
  }
}
```

This code is used in section 3.

29. ⟨Lay the doubled packing in its torus 29⟩ $\equiv$

```
t := vec{2 * l, 2 * m, 2 * n}
var all [][]vec
for _, c := range corners {
  for _, sh := range []vec{{}, {l, m, n}} {
    start := vec{c.x + sh.x, c.y + sh.y, c.z + sh.z}
    all = append(all, tripod(start, vec{l, m, n}, t))
  }
}
used, clashes := lay(all)
want := 2 * r * (1 + l + m + n)
mark := "ok"
if len(used) ≠ want ∨ clashes ≠ 0 {
  mark = "FAILS"
}
fmt.Printf("_%d%d%d: r=%2d, %2d tripods in %2dx%2dx%2d, %3d cubies (want %3d), " +
  "%d clashes; density %.6f\n",
  l, m, n, r, len(all), t.x, t.y, t.z, len(used), want, clashes,
  float64(want)/float64(8 * l * m * n), mark)
```

This code is used in section 28.

30. Sweeping the tori. Answer 346(b) asks in passing whether 7/9 is optimum. A shifted packing that repeats with period (X, Y, Z) is the same thing as a packing of the $X \times Y \times Z$ torus, so the question can at least be asked of every small torus in turn: how many (l, m, n) -tripods fit? The encoding is the one part (e) uses, except that now every cubie of the torus is a candidate corner and all of them are secondary.

⟨ Functions 5 ⟩ +≡

```
func torusProblem(leg, t vec, k int) io.Reader {
    r, w := io.Pipe()
    go func() {
        defer w.Close()
        fmt.Fprintf(w, "%d:%d|#_|", k, k)
        for x := range t.x {
            for y := range t.y {
                for z := range t.z {
                    fmt.Fprintf(w, "%s", name(vec{x, y, z}))
                }
            }
        }
        fmt.Fprintln(w)
        for x := range t.x {
            for y := range t.y {
                for z := range t.z {
                    fmt.Fprint(w, "#")
                    for _, q := range tripod(vec{x, y, z}, leg, t) {
                        fmt.Fprintf(w, "%s", name(q))
                    }
                    fmt.Fprintln(w)
                }
            }
        }
    }()
    return r
}
```

31. No side of the torus may be as short as a leg, or the leg would wrap back onto its own corner and the piece would not be a copy of the tripod at all. Counting down from the most that could fit by volume alone gives the maximum.

⟨ Sweep the tori 31 ⟩ ≡

```
fmt.Printf("torus_sweep_for_(%d,%d,%d)-tripods,_volume_up_to_%d\n",
    sweep.x, sweep.y, sweep.z, *vol)
size := 1 + sweep.x + sweep.y + sweep.z
best := 0.0
for X := sweep.x + 1; X * (sweep.y + 1) * (sweep.z + 1) ≤ *vol; X++ {
    for Y := sweep.y + 1; X * Y * (sweep.z + 1) ≤ *vol; Y++ {
        for Z := sweep.z + 1; X * Y * Z ≤ *vol; Z++ {
            ⟨ Find the most tripods this torus holds 32 ⟩
        }
    }
}
fmt.Printf("_best_density_over_the_sweep: %.6f\n", best)
```

This code is used in section 3.

32. $\langle$ Find the most tripods this torus holds [32](#) $\rangle \equiv$

```

t := vec{X, Y, Z}
V := X * Y * Z
k := V/size + 1
for k > 0 {
    sol, slow := first(torusProblem(sweep, t, k), budget)
    if slow {
        fmt.Printf("%2dx%2dx%2d: timed out at %d\n", X, Y, Z, k)
        break
    }
    if sol ≠ Λ {
        break
    }
    k --
}
d := float64(k * size)/float64(V)
flag := ""
if d > best {
    best, flag = d, "new best so far"
}
fmt.Printf("%2dx%2dx%2d volume %4d: %3d tripods, density %4d/%-4d = %.6f s\n",
    X, Y, Z, V, k, k * size, V, d, flag)

```

This code is used in section [31](#).

33. Index.

a: [19](#).
all: [11](#), [15](#), [17](#), [19](#), [21](#), [29](#).
append: [6](#), [9](#), [10](#), [11](#), [14](#), [15](#), [17](#), [19](#), [25](#), [29](#).
Atoi: [8](#).
b: [19](#).
Background: [9](#).
bad: [11](#), [12](#), [15](#).
below: [12](#).
best: [25](#), [31](#), [32](#).
bool: [7](#), [9](#), [16](#), [22](#), [25](#).
budget: [2](#), [9](#), [25](#), [27](#), [28](#), [32](#).
byte: [14](#), [15](#).
c: [6](#), [7](#), [10](#), [12](#), [15](#), [16](#), [17](#), [19](#), [23](#), [24](#), [29](#).
cancel: [9](#).
cells: [1](#), [9](#).
ch: [14](#), [15](#).
clashes: [7](#), [11](#), [15](#), [17](#), [19](#), [21](#), [29](#).
Close: [22](#), [30](#).
context: [9](#).
corner: [15](#), [19](#).
corners: [17](#), [28](#), [29](#).
cs: [15](#), [16](#).
ctx: [9](#).
d: [10](#), [12](#), [32](#).
d1: [27](#).
d2: [27](#).
Dance: [9](#).
den: [21](#).
drop: [12](#).
Duration: [2](#), [9](#), [25](#).
Err: [9](#).
fam: [10](#), [12](#).
far: [22](#), [23](#).
filled: [20](#), [21](#).
first: [9](#), [25](#), [32](#).
flag: [2](#), [32](#).
float64: [15](#), [17](#), [21](#), [29](#), [32](#).
fnt: [2](#), [8](#), [11](#), [12](#), [14](#), [15](#), [17](#), [19](#), [21](#), [22](#), [23](#), [24](#), [26](#),
[27](#), [28](#), [29](#), [30](#), [31](#), [32](#).
force: [22](#), [23](#), [25](#).
found: [15](#).
Fprint: [22](#), [24](#), [30](#).
Fprintf: [22](#), [23](#), [24](#), [30](#).
Fprintln: [22](#), [24](#), [30](#).
here: [12](#).
hi: [20](#).
i: [6](#), [10](#), [11](#).
in: [16](#).
inside: [20](#), [21](#).
Int: [2](#).
int: [4](#), [7](#), [10](#), [12](#), [15](#), [22](#), [25](#), [26](#), [30](#).
Ints: [15](#).
io: [9](#), [22](#), [30](#).
j: [6](#).
k: [6](#), [30](#), [32](#).
key: [27](#).
l: [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [28](#), [29](#).
lay: [7](#), [11](#), [15](#), [17](#), [19](#), [29](#).
layer: [14](#).
layerA: [10](#), [11](#), [12](#).
leg: [6](#), [30](#).
legs: [2](#).
len: [11](#), [12](#), [15](#), [16](#), [17](#), [21](#), [29](#).
lo: [20](#).
m: [10](#), [11](#), [12](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [28](#), [29](#).
main: [1](#).
mark: [27](#), [29](#).
match: [15](#), [16](#).
Millisecond: [27](#).
mins: [2](#).
Minute: [2](#).
miss: [12](#).
mode: [2](#), [3](#).
N: [10](#), [11](#), [12](#), [19](#), [20](#), [21](#).
n: [10](#), [11](#), [12](#), [19](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [28](#), [29](#).
name: [8](#), [22](#), [23](#), [24](#), [30](#).
NewMCC: [9](#).
note: [21](#).
Now: [27](#).
num: [21](#).
opt: [25](#).
Option: [9](#).
p: [7](#), [8](#), [12](#), [19](#).
Parse: [2](#).
parse: [8](#), [25](#).
piece: [14](#), [15](#).
pieces: [7](#).
Pipe: [22](#), [30](#).
podProblem: [22](#), [25](#).
printedB: [13](#), [14](#).
Printf: [11](#), [12](#), [15](#), [17](#), [21](#), [27](#), [29](#), [31](#), [32](#).
Println: [11](#), [14](#), [17](#), [19](#), [26](#), [28](#).
q: [16](#), [24](#), [30](#).
r: [9](#), [22](#), [28](#), [29](#), [30](#).

r1: [27](#).
r2: [27](#).
Reader: [9](#), [22](#), [30](#).
res: [9](#).
rOf: [25](#), [27](#), [28](#).
Round: [27](#).
row: [14](#).
s: [6](#), [8](#), [9](#), [15](#).
sh: [29](#).
Since: [27](#).
size: [31](#), [32](#).
slow: [9](#), [25](#), [32](#).
slow1: [27](#).
slow2: [27](#).
sol: [9](#), [25](#), [32](#).
Solutions: [9](#).
sort: [15](#).
Split: [8](#).
Sprintf: [8](#), [21](#), [27](#).
Sscanf: [2](#).
start: [29](#).
strconv: [8](#).
String: [2](#).
string: [8](#), [13](#), [26](#).
strings: [8](#).
sweep: [2](#), [31](#), [32](#).
syms: [15](#).
t: [5](#), [6](#), [10](#), [14](#), [16](#), [22](#), [25](#), [29](#), [30](#), [32](#).
t0: [27](#).
time: [2](#), [9](#), [25](#), [27](#).
top: [2](#), [11](#), [12](#), [19](#).
torusProblem: [30](#), [32](#).
tripod: [6](#), [10](#), [16](#), [17](#), [19](#), [24](#), [29](#), [30](#).
used: [7](#), [11](#), [15](#), [17](#), [19](#), [20](#), [29](#).
V: [32](#).
v: [5](#), [8](#).
vec: [2](#), [4](#), [5](#), [6](#), [7](#), [8](#), [10](#), [11](#), [14](#), [15](#), [16](#), [17](#), [19](#), [20](#),
[22](#), [23](#), [24](#), [25](#), [29](#), [30](#), [32](#).
vol: [2](#), [31](#).
w: [22](#), [23](#), [24](#), [30](#).
want: [26](#), [27](#), [29](#).
WithContext: [9](#).
WithTimeout: [9](#).
wrap: [5](#), [6](#).
X: [31](#), [32](#).
x: [2](#), [4](#), [5](#), [6](#), [8](#), [12](#), [14](#), [15](#), [20](#), [23](#), [24](#), [29](#), [30](#), [31](#).
Y: [31](#), [32](#).
y: [2](#), [4](#), [5](#), [6](#), [8](#), [12](#), [14](#), [15](#), [20](#), [23](#), [24](#), [29](#), [30](#), [31](#).

Z: [31](#), [32](#).

z: [2](#), [4](#), [5](#), [6](#), [8](#), [12](#), [14](#), [15](#), [20](#), [23](#), [24](#), [29](#), [30](#), [31](#).

- ⟨ Ask whether the tripod at c is exactly the piece 16 ⟩ Used in section 15
- ⟨ Check the diagonal packings 19 ⟩ Used in section 3
- ⟨ Check the thirteen corners of answer 346(c) 17 ⟩ Used in section 3
- ⟨ Check the two facts the construction rests on 12 ⟩ Used in section 11
- ⟨ Do what the mode asks 3 ⟩ Used in section 1
- ⟨ Double a pod packing as part (d) says 28 ⟩ Used in section 3
- ⟨ Evaluate $r(l, m, n)$ 26 ⟩ Used in section 3
- ⟨ Evaluate one cuboid both ways 27 ⟩ Used in section 26
- ⟨ Find the most tripods this torus holds 32 ⟩ Used in section 31
- ⟨ Functions 5 ⟩ Used in section 1
- ⟨ Lay the doubled packing in its torus 29 ⟩ Used in section 28
- ⟨ Measure the density inside a window 20 ⟩ Used in section 19
- ⟨ Name the corner of every printed piece 15 ⟩ Used in section 14
- ⟨ Name the cubies that stay secondary 23 ⟩ Used in section 22
- ⟨ Read the command line 2 ⟩ Used in section 1
- ⟨ Read the packing printed in answer 346(b) 14 ⟩ Used in section 3
- ⟨ Report the density of the diagonal packing 21 ⟩ Used in section 19
- ⟨ Sweep the tori 31 ⟩ Used in section 3
- ⟨ Tile space with $(1, m, n)$ -tripods 11 ⟩ Used in section 3
- ⟨ Types 4 ⟩ Used in section 1
- ⟨ Write one option for each pod 24 ⟩ Used in section 22

Tripods

	Section	Page
Introduction	1	1
Tripods, pods, and tori	4	3
Part (a): tiling space with $(1, m, n)$ -tripods	10	5
Part (b): the packing that is printed, and $7/9$	13	7
Part (c): thirteen tripods, and a better packing	17	9
The diagonal packing	18	10
Part (e): how many pods fit in a cuboid	22	12
Part (d): doubling a pod packing	28	15
Sweeping the tori	30	16
Index	33	18